FINAL PROJECT SUBMISSION REPORT

AI Powered Web Application

Hybrid AI Study Studio: three local specialists plus OpenAI GPT-4o mini

Course: CS 85 - PHP Programming
Instructor: Vicky Seno
Student: Siarhei

Application: Laravel 13 / PHP 8.4
Local AI Models (LM Studio): ***Qwen 3.6** 35B A3B*, ***Qwen 3** Coder Next*, ***OpenAI** GPT-OSS 120B*
Online AI Model (OpenAI API): ***OpenAI GPT-4o mini***
Application Video Demo (GitHub Pages): https://sergehall.github.io/cs85-php-programming/
GitHub: https://github.com/SergeHall/cs85-php-programming

PROJECT RESULT **A production-style, authenticated Laravel AI workspace that routes each conversation to one of three private local models or the OpenAI GPT-4o mini online model.**

1. Project Overview

The final project expands the focused Module 12A OpenAI integration into a complete multi-provider AI learning workspace. Module 12A demonstrated the essential API workflow: Laravel accepts a prompt, sends a server-side request to OpenAI GPT-4o mini, receives the generated content, and returns it to the browser without exposing the API key. The final AI Study Studio preserves this secure integration and extends it with local inference, specialized assistant modes, persistent multi-turn conversations, streaming responses, provider monitoring, retry support, and sanitized Markdown rendering.

The workspace offers four task-oriented AI specialists with different levels and areas of expertise:

- **General Tutor - Qwen 3.6 35B A3B:** A local LM Studio model for concept explanations, quizzes, and general programming questions.
- **Coding Assistant - Qwen 3 Coder Next:** A local coding-focused model for implementation guidance, debugging, code review, and technical problem-solving.
- **Architecture Advisor - OpenAI GPT-OSS 120B:** The largest local model, used for system design, architectural decisions, maintainability reviews, and trade-off analysis.
- **OpenAI Online - GPT-4o mini:** A cloud model accessed through the OpenAI API for fast responses, polished drafts, summaries, and live third-party API integration.

Each model card combines the specialist's purpose, provider, runtime profile, model identifier, live connection status, and the action used to create a conversation. This design allows the user to

understand both the model's expertise and where the prompt will be processed before beginning a chat. Local modes keep inference within LM Studio on the user's computer, while the online mode sends prompts to OpenAI only after the user explicitly selects GPT-4o mini.

After a conversation is created, Laravel stores its selected mode, provider, and model so that later messages remain pinned to the same specialist. User and assistant messages are persisted as conversation history, while same-origin Server-Sent Events deliver response fragments incrementally to the browser. The interface renders sanitized Markdown and formatted code, reports progress and errors, and supports safe retries when a provider request fails.

The workspace also performs live health checks for both LM Studio and the OpenAI API. These checks confirm model availability and display non-sensitive latency information without exposing API credentials. In the recorded browser evidence, all three local models and the OpenAI online model are connected, demonstrating that the application operates as a functioning hybrid AI system rather than a standalone API form.

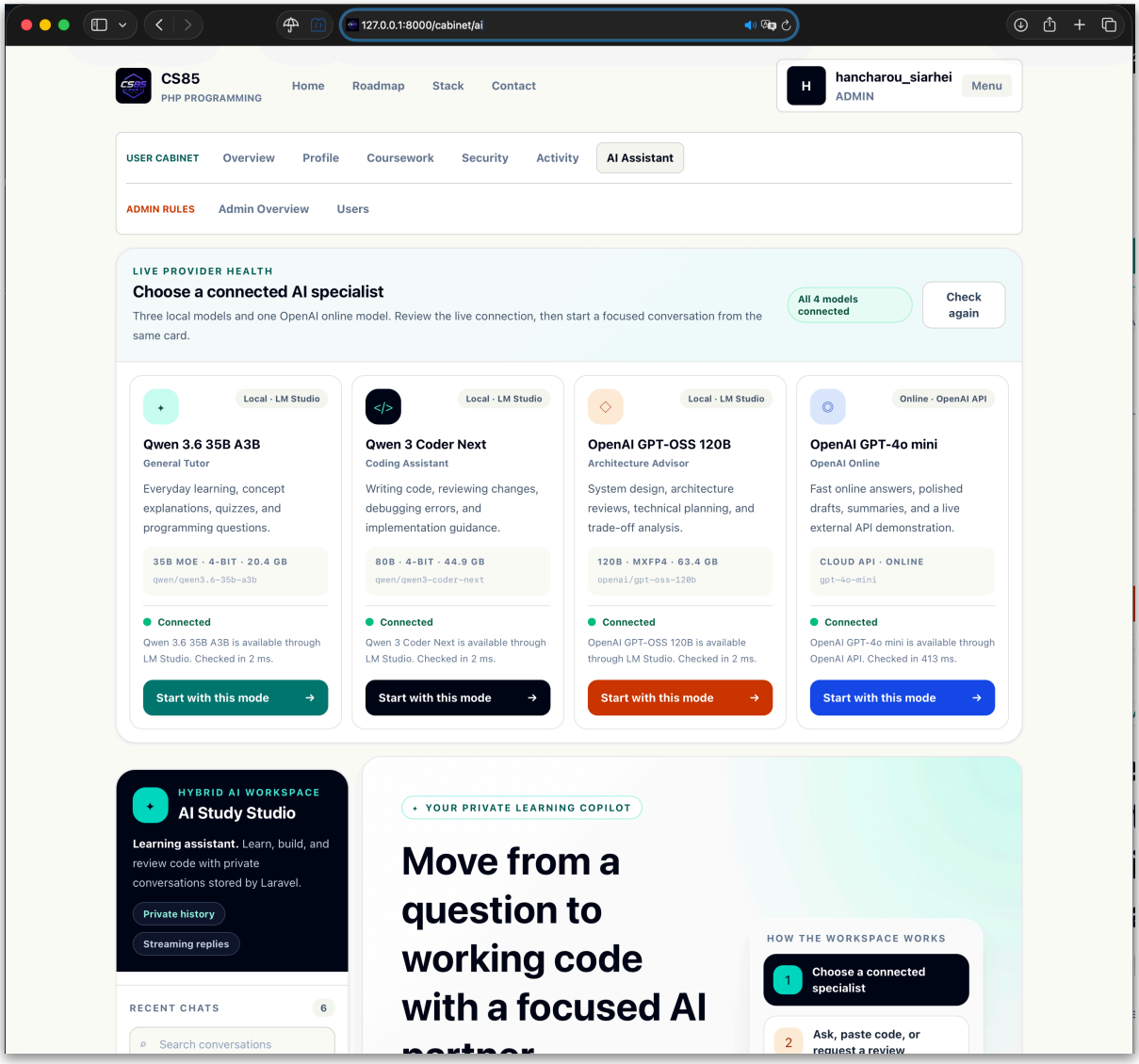


Figure 1. AI Cabinet model selector and live provider connection status.

Delivered Capabilities

Hybrid AI inference: The application connects three local LM Studio specialists Qwen 3.6 35B A3B for general learning, Qwen 3 Coder Next for programming assistance, and OpenAI GPT-OSS 120B for architecture analysis plus GPT-4o mini through the OpenAI API for online responses.

Persistent conversations: Laravel stores each conversation, user and assistant messages, selected mode, provider, and model identifier. This allows later messages to continue with bounded conversation context.

Streaming chat experience: Same-origin SSE delivers responses incrementally. Markdown and code blocks are safely rendered, while loading, completion, tool, retry, and error states are shown directly in the interface.

Live provider monitoring: The workspace checks LM Studio and OpenAI availability, reports connection status and latency, and confirms which models are ready without exposing API keys or other credentials.

Explicit and private routing: Every conversation remains pinned to the model selected by the user. The application does not silently transfer local prompts to OpenAI, preserving clear control over privacy and provider usage.

2. Architecture and Model Integration

```
Authenticated and verified browser
    -> Laravel cabinet routes
        -> authentication, verification, CSRF, and rate limits
    -> Form Requests
        -> mode allowlist and prompt validation
    -> AiAssistantController
        -> conversation ownership and SSE response boundary
    -> AiConversationService
        -> ModelRouter
        -> PromptBuilder
        -> bounded Laravel-owned history
        -> AiToolRegistry
    -> AiProviderInterface
        -> RoutedAiProvider
            -> LM Studio
                -> Qwen 3.6 35B A3B — General Tutor
                -> Qwen 3 Coder Next — Coding Assistant
                -> OpenAI GPT-OSS 120B — Architecture Advisor
            OR
            -> OpenAI API
                -> GPT-4o mini — OpenAI Online
    -> streamed delta, status, complete, or error events
    -> authenticated browser
```

Persistence is coordinated by AiConversationService, not by either provider:

```
ai_conversations
    -> selected mode, provider, model, title, and owner

ai_messages
    -> persistent user and assistant conversation history

ai_requests
    -> status, latency, token counts, provider, model, and error code
```

Layer Responsibilities

Interface layer: Blade renders the authenticated AI workspace, model cards, conversation history, provider-health indicators, and chat controls. JavaScript creates conversations, submits messages through same-origin requests, consumes the SSE stream, updates partial responses, displays safe errors, and supports retry and conversation deletion.

Web boundary: Authenticated and verified cabinet routes protect the workspace. Form Requests validate the selected mode and prompt length, while rate-limit middleware controls status checks and AI requests. AiAssistantController verifies conversation ownership, delegates work to the application service, and returns properly configured streamed responses.

Application layer: AiConversationService coordinates the complete request lifecycle: storing messages, building bounded context, invoking providers, processing tools, persisting responses, handling retries, and recording telemetry. ModelRouter, PromptBuilder, AiToolRegistry, and AiMarkdownRenderer provide focused routing, prompt, tool, and safe-rendering responsibilities.

Provider layer: AiProviderInterface defines one transport-independent streaming contract. RoutedAiProvider selects LmStudioProvider or OpenAiProvider using the provider already stored on the conversation. Both integrations use the same OpenAI-compatible request and streaming structure while keeping their credentials and timeouts separate.

Persistence layer: Laravel Eloquent stores user-owned conversations, user and assistant messages, selected mode/provider/model values, and request telemetry. AiRequest records contain operational metadata such as status, latency, token counts, and error codes—not API credentials or duplicated prompt content.

Configured AI Specialists

Mode	Provider / model	Primary purpose
General Tutor	LM Studio Qwen 3.6 35B A3B	Concept explanations, quizzes, and general programming questions.
Coding Assistant	LM Studio Qwen 3 Coder Next	Code generation, review, debugging, and implementation guidance.
Architecture Advisor	LM Studio OpenAI GPT-OSS 120B	System design, planning, trade-offs, and maintainability reviews.
OpenAI Online	OpenAI API GPT-4o mini	Fast online answers, polished drafts, summaries, and live API evidence.

OpenAI GPT-4o mini Integration

The online mode is not called from the browser. Laravel reads OPENAI_API_KEY and OPENAI_MODEL from the server environment through `config/ai.php`. OpenAiProvider reuses the OpenAI-compatible transport to send an authenticated POST /v1/chat/completions request with streaming enabled. Laravel parses the provider stream, emits its own safe SSE events, and stores the completed assistant message. The model is configurable and defaults to gpt-4o-mini.

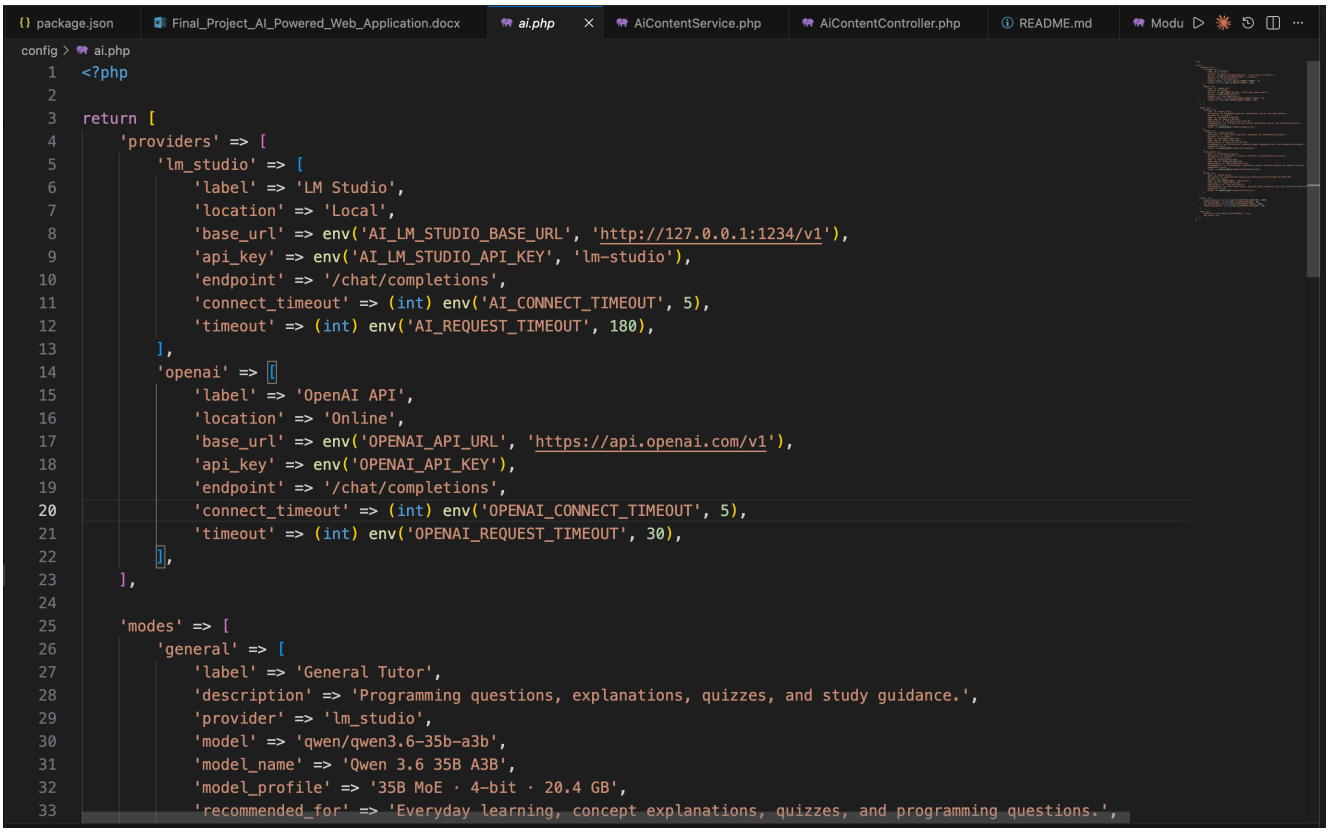


Figure2. Server-side provider routing and OpenAI configuration code.

3. Request Lifecycle, Security, and Verification

Request Lifecycle

1. An authenticated and verified user selects one of the four modes defined in config/ai.php.
   Laravel creates a conversation and stores its mode, provider, and model so routing cannot change during later turns.
2. SubmitAiMessageRequest trims and validates the prompt. Laravel verifies conversation ownership, stores the user message, generates the initial conversation title, and creates an AiRequest telemetry record with processing status.
3. PromptBuilder adds the mode-specific system prompt to the bounded Laravel-owned message history. The request also includes the configured temperature, output-token limit, and definitions for the approved read-only course tools.
4. RoutedAiProvider reads the provider stored on the conversation and delegates the request to either LM Studio or OpenAI. Provider chunks are emitted as SSE delta events, with periodically refreshed sanitized Markdown HTML.
5. If the model requests a tool, Laravel emits a status event, validates and executes the allowlisted tool, and sends its result through a second provider pass without exposing unrestricted application access.
6. On success, Laravel stores the assistant Markdown, updates token and latency telemetry, and emits a complete event containing sanitized HTML. Failures update the request to failed and emit a safe error event. Retries reuse the original user message without creating a duplicate.

Security and Failure Handling

AI credentials remain in the ignored .env file and are used only by Laravel. The browser never receives the OpenAI key or communicates directly with OpenAI or LM Studio. The /cabinet/ai workspace requires an authenticated, verified user, uses CSRF protection and rate limits, and scopes every conversation to its owner.

Users select one of the four modes defined in config/ai.php; they cannot provide arbitrary models, providers, or API URLs. Prompt length, conversation history, output tokens, and request frequency are limited. AI Markdown is sanitized, unsafe HTML and links are rejected, and course tools are read-only, allowlisted, and schema-validated.

Provider timeouts, unavailable models, rejected credentials, and interrupted streams become safe retryable errors. The application never silently switches from LM Studio to OpenAI. Private prompts reach GPT-4o mini only when the user explicitly selects OpenAI Online mode.

Testing and Demonstration Evidence

Feature and unit tests cover access control, routing, ownership, streaming events, retries, model health, providers, and tools. OpenAI and LM Studio HTTP boundaries are replaced with `Http::fake()` so tests are deterministic, spend no API credits, and require no live model. Node tests verify SSE parsing and connection summaries.

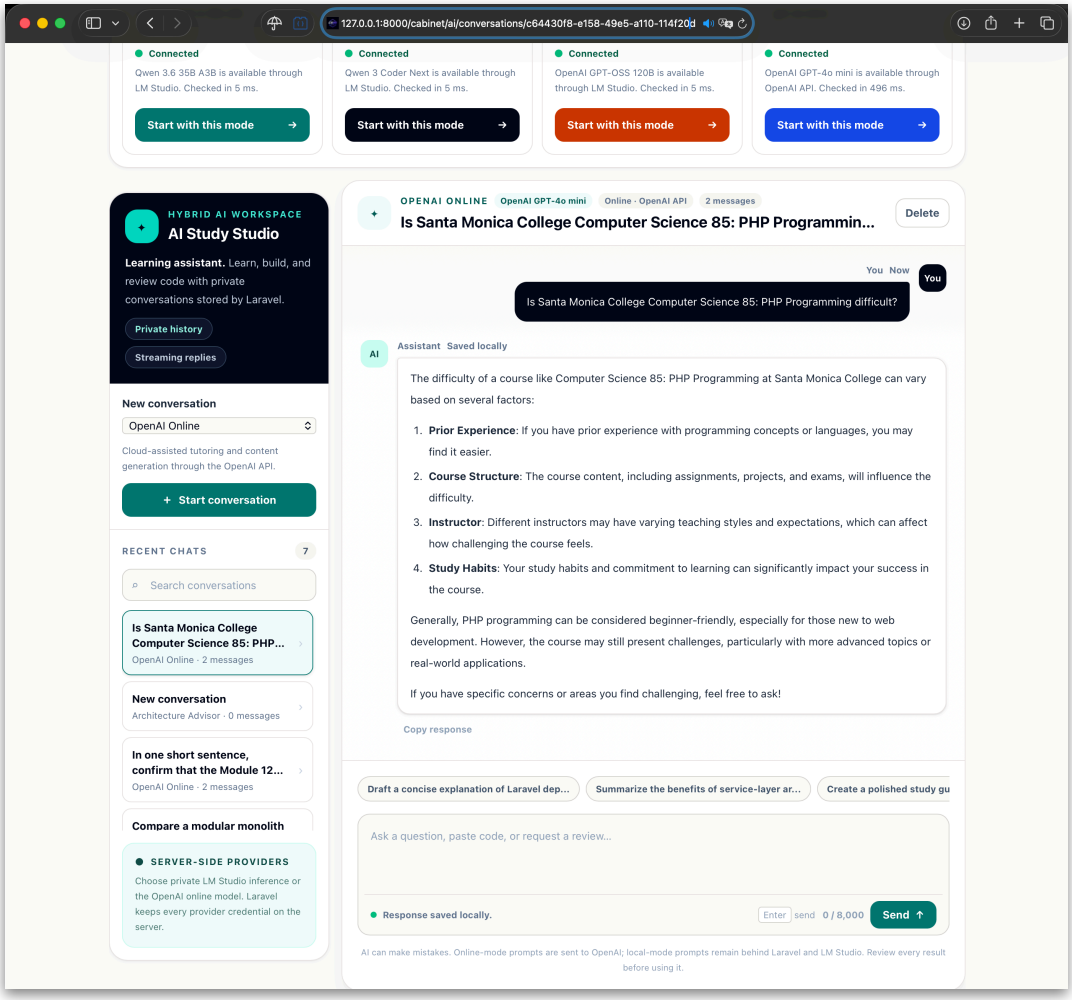


Figure 3. Completed AI conversation with a visible streamed response.

Conclusion

The project fulfills the final objective by integrating the OpenAI API through a secure Laravel service boundary and expanding that integration into a complete hybrid AI application. OpenAI GPT-4o mini serves as the online model for fast responses, polished drafts, summaries, and verifiable third-party API functionality. The API key remains on the Laravel server and is never exposed to the browser. Requests are validated, rate-limited, monitored, and handled with safe error responses.

The application also provides three local AI specialists through LM Studio, each selected for a different level and area of expertise. Qwen 3.6 35B A3B operates as the General Tutor for explanations, quizzes, and everyday programming questions. Qwen 3 Coder Next provides deeper coding specialization for code generation, review, debugging, and implementation guidance. OpenAI GPT-OSS 120B serves as the Architecture Advisor for complex system design, technical planning, maintainability analysis, and engineering trade-offs. These models run locally, giving the application a private inference path that does not require sending conversation content to an external provider.

Laravel explicitly routes every conversation to the selected provider and model while retaining responsibility for authentication, authorization, prompts, conversation ownership, persistent multi-turn history, streaming, retries, read-only tools, and request telemetry. Live provider-health checks confirm whether each local model and the OpenAI online model are available before a conversation begins. Automated tests use fake providers and HTTP responses, allowing routing, security, failure handling, and API behavior to be verified without depending on live services or consuming OpenAI credits.

The final result is more than a standalone API demonstration. It is a working AI Study Studio that combines the privacy and specialized depth of local models with the accessibility and speed of an online OpenAI model. This hybrid architecture demonstrates how one Laravel application can support multiple AI capabilities while maintaining clear boundaries, secure credential handling, reliable persistence, and a focused user experience.